

2026-09-25

Arity: A Standalone Language Server for R Package Development

R Consortium ISC proposal

Johan Larsson 

Department of Mathematical Sciences, University of Copenhagen

Executive Summary

R package development involves working across source files, dependencies, and documentation. I created [Arity](#) to help developers navigate and check this code from their editor without running R. Arity is an MIT-licensed language server written in Rust, with an integrated formatter and linter. It supports VS Code, Positron, Neovim, Zed, and other editors through the language server protocol (LSP).

This project will improve Arity's handling of shared documentation and function names, and provide function signatures and help for packages that are not installed locally, helping developers inspect unfamiliar code before installing its dependencies. I will publish the metadata and extraction pipeline for reuse by other R tools, and test the improvements in complete editing workflows. I ask for USD 10,000 to fund 125 hours of labor.

Signatories

Project team

I am Johan Larsson, Arity's maintainer and a postdoctoral researcher in statistics and machine learning at the University of Copenhagen's Department of Mathematical Sciences. I also develop and maintain several R packages, including [eulerr](#), [SLOPE](#), and [qualpalr](#).

Contributors

I will carry out the work myself.

Consulted

The proposal draft was published in a [public GitHub repository](#) and I invited user feedback through social media and the issue tracker for Arity. I'm grateful for feedback from Jan Hartman and Tatsuya Shima (maintainer of [arf](#), [r-documentation-rs](#), and [mini-roxygen](#)), whose suggestions helped clarify and improve the proposal.

The Problem

There are three language servers for R: [languageserver](#), [Ark](#), and Arity. [languageserver](#) is a mature R package that runs in an R session. Ark embeds its language server in an R kernel and powers R support in Positron. Arity, meanwhile, is a standalone language server that works without an R session or Jupyter kernel. Adapting Ark to Arity's design would have required extracting the server and replacing its dependencies on the R runtime, which is what motivated the development of Arity.

Yet Arity still has limitations that affect package development. It can, for instance, misinterpret shared roxygen documentation, miss undocumented parameters, and handle references and call hierarchy inconsistently. Renaming can also produce invalid R syntax, for example when a function is used as an infix operator.

Arity also needs better support for dependencies that are not installed locally. It provides signatures and help for installed packages, but only exported names for others. Richer metadata would provide that assistance without installation and enable linting of deprecated and defunct functions. The editor and command-line interface (CLI) also need to share cached metadata when downloads are disabled.

The Proposal

Overview

I will improve Arity's analysis of package code, extend its metadata for dependencies, and test these changes together in complete editing workflows.

Detail

Minimum Viable Product

Language-server reliability I will improve Arity's handling of shared and inherited roxygen documentation. References, renaming, and call hierarchy will handle quoted function names consistently. Renaming will produce valid R syntax, including in infix calls, or Arity will refuse the operation.

Package knowledge for editor assistance I will publish versioned indexes for at least 100 CRAN packages meeting the quality criteria below, including packages outside the existing bundle and dependencies in the evaluation workflows. The indexes will contain exported names, symbol kinds, recoverable arguments and defaults, and help. They will also record deprecated and defunct functions identified from explicit declarations, with replacements where available. Arity will use these indexes for completion, hover information, signature help, and linting without local package installation.

I will document the schema, automate weekly updates, and add a prefetch command for downloading indexes in bulk. The editor and CLI will use the same cached metadata and work with downloads disabled. Network access will remain an explicit user choice. Each index will record the package version and how the metadata was extracted.

Validation and documentation I will test complete editing workflows on ten packages with cross-file references, imports, shared documentation, and nonstandard evaluation. I will update the guides for editors and continuous integration (CI), explaining how to use Arity alongside other R tools.

Architecture

Arity’s metadata harvester already reads serialized R objects and installed-package help without running R. Its binary parsing and Rd decoding overlap with [r-documentation-rs](#). I will coordinate with its maintainer on consolidating these readers in `r-documentation-rs`, reducing duplication and making improvements available to other R tools. Arity will continue to handle package discovery, export resolution, metadata indexing, caching, and editor integration.

As of September 2026, Arity bundles export names for over 500 packages and has a workflow to refresh them weekly. The remote client already supports downloads, disk caching, and versioned indexes, but accepts only exported names. I will add a pipeline to publish richer indexes, extend the client to supply signatures and help, and make cached exports available to the CLI. Local indexes will take precedence, and Arity will show the package version when it uses a remote index.

Assumptions

Static analysis cannot determine what every name refers to at runtime. Arity will refuse uncertain rewrites and withhold deprecation diagnostics when function resolution or the indexed deprecation status is uncertain. I will document unsupported cases. The publishing pipeline will initially target current package releases. An available index may describe a different version from a project’s dependency. When Arity cannot establish that the versions match, deprecation findings will identify the indexed release and remain advisory, without causing the CLI to fail.

I will record missing fields and failed extractions, and older metadata will retain the version of the package it describes. Backfilling historical releases, selecting indexes from project lockfiles, argument-specific deprecations, and signatures that require running R are outside this project’s scope.

External Dependencies

For the pilot, I will extract metadata from unpacked [R-universe binary packages](#), falling back on packages installed in CI when needed. I will check the complete export lists and samples of signatures and help against the same package versions in R, and use `roxygen2` to check documentation behavior. The source packages and extracted data will retain their version and license information.

Project Plan

Start-Up Phase

The first milestone (M1) will begin in December 2026, after the grant agreement. I will pin versions of ten evaluation packages, define their editing workflows, and record baseline diagnostics and response times. I will reproduce the analysis problems, starting with existing cases from `future`, and document the setup for comparison with the final results.

Initial extraction checks in September 2026 found the required fields for every exported function in glue, withr, R6, and cli. For magrittr, coverage was 16 of 42 functions, largely because primitive aliases lack formal arguments. The checks also found S3 methods incorrectly listed as exports. The metadata pilot will use glue, withr, R6, and cli, along with tynitable to test a package outside the existing bundle. I will also use magrittr to test incomplete indexes. I will fix the S3 export errors during M1, before publication.

During M1, I will identify the overlapping code, missing APIs, and tests needed to consolidate the readers in r-documentation-rs. I will also test extraction of deprecation declarations and define the supported cases before M2. If full consolidation exceeds the available hours, I will retain the necessary parts of Arity’s extractor and defer the remaining consolidation work.

Before M2, five pilot packages must pass the metadata quality criteria below. A prototype using Arity’s existing client must provide signatures and help without local installation for packages both inside and outside the existing bundle. If the pilot fails to meet these requirements within budget, I will agree on a reduced scope with the ISC, prioritizing metadata features and offline access.

Technical Delivery

Milestone	Target date	Deliverables	Hours	Cost (USD)
M1: Baseline and pilot	December 31, 2026	Baseline, metadata prototype, consolidation plan, quality checks, regression cases	30	2,400
M2: Implementation	January 31, 2027	Reliability fixes, rich metadata publication and integration, prefetch, shared offline cache	65	5,200
M3: Validation and release	February 28, 2027	Workflow validation, annotated comparison, guides, release, report, maintenance instructions	30	2,400
Total			125	10,000

The estimates include regression tests, with eight hours in M2 set aside for unexpected work. If the grant agreement is delayed, I will agree on revised milestone dates with the ISC before work begins.

Other Aspects

Code and tests will use Arity’s MIT license. Evaluation sources and examples will retain applicable notices. I will publish the schema, code for the publishing pipeline, documentation, and results, along with instructions for rebuilding or mirroring the dataset.

I will invite package authors and tooling developers to try a prerelease and write R Consortium blog posts announcing the project and sharing results, examples, and limitations.

Budget & Funding Plan

I intend to receive the grant personally. The requested USD 10,000 will fund 125 hours of my time at USD 80 per hour. The budget assumes no other funding or contributors.

With partial funding, I would publish indexes for at least 50 CRAN packages meeting the same quality criteria and evaluate five packages. This would still include the metadata features and offline support described above, plus safe infix renaming. I would defer the other reliability fixes and agree on the hours, funding, and dates with the ISC before accepting partial funding.

Success

Definition of Done

The project will be complete when I have published a tagged Arity release and met the following criteria:

Language-server reliability Tests show that Arity resolves shared documentation, reports undocumented parameters, and handles references and call hierarchy consistently for quoted function names. For supported quoted-name and infix cases, renaming updates definitions and statically resolved references, leaves unrelated bindings unchanged, and produces valid R syntax. Tests also verify that Arity refuses uncertain rewrites.

Package knowledge Indexes for at least 100 CRAN packages are published, with a documented schema and weekly updates. Export lists must match R's results for the same versions. Sampled arguments, defaults, and help must match or be withheld. After withholding incorrect or uncertain fields, each qualifying index must provide arguments, defaults, and help for at least 80% of exported functions. Missing fields will be reported.

Tests show completion details, hover information, and signature help without local installation, including packages outside the bundle. They also verify diagnostics for deprecated and defunct functions, covering supported declarations, local shadowing, and advisory behavior when dependency versions cannot be matched. The same tests pass offline after prefetch. With identical source, configuration, and cached metadata, name resolution and diagnostics agree between the editor and CLI.

Validation and documentation I have published workflows and before-and-after results for ten packages, including checks in VS Code and Neovim, along with editor and CI guides and maintenance instructions. The release passes the existing workspace and corpus checks with no unexplained regressions.

Measuring Success

The report will compare diagnostics and response times before and after the project using the ten package workflows. I will classify sampled diagnostic findings as defects, false positives, or unresolved cases, and report remaining limitations and any new regressions. For metadata, I will report coverage, accuracy, how current the data is, and extraction failures, with deprecation coverage reported separately. The report will document the versions, configuration, and measurement setup so others can reproduce the comparison.

Future Work

Future work could cover more packages, select exact versions from a project's lockfile, support the CRAN Archive with package metadata from [ALLPACKAGES](#), and use runtime tracing.